s443 group project

AroraLinMajumdarRathore

2026-03-26

Step 1: load and clean data set

- load libraries

```
library(readr)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(tidyr)
library(tibble)
library(tidyverse)
```

```
## Warning: package 'ggplot2' was built under R version 4.3.2
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✔ forcats   1.0.0     ✔ purrr     1.0.2
## ✔ ggplot2   3.5.1     ✔ stringr   1.5.0
## ✔ lubridate 1.9.3
```

```
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflic
ts to become errors
```

```
library(lubridate)
library(forecast)
```

```
## Warning: package 'forecast' was built under R version 4.3.3
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```
library(janitor)
```

```
## Warning: package 'janitor' was built under R version 4.3.3
```

```
##
## Attaching package: 'janitor'
##
## The following objects are masked from 'package:stats':
##
##     chisq.test, fisher.test
```

- Note: all data starts from Jan 1997 to Dec 2025. Data obtained from Statistics Canada.
    - Target time series:
        - unemployment rate (unit: percentage)
    - External variables:
        - employment by industry (unit: 1000 people)
        - GDP by industry (unit: 1,000,000 cad)
- Reference:
    - https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=1410002201 (https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=1410002201) : Employment data per industry
    - https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=1410028701 (https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=1410028701) : Labour Force Characteristics (Unemployment rate)
    - https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=3610043401 (https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=3610043401) : GDP Data per industry

```
eda_data <- read_csv("eda_data.csv")
```

```
## Rows: 348 Columns: 4
## ── Column specification ────────────────────────────────────────────────────────
## Delimiter: ","
## dbl  (3): unemployment_rate, total_employment, all_gdp
## date (1): month
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Step 2.1: Explanatory Data Analysis (Summary table and Plots):

- Summary table

```
summary(eda_data)
```

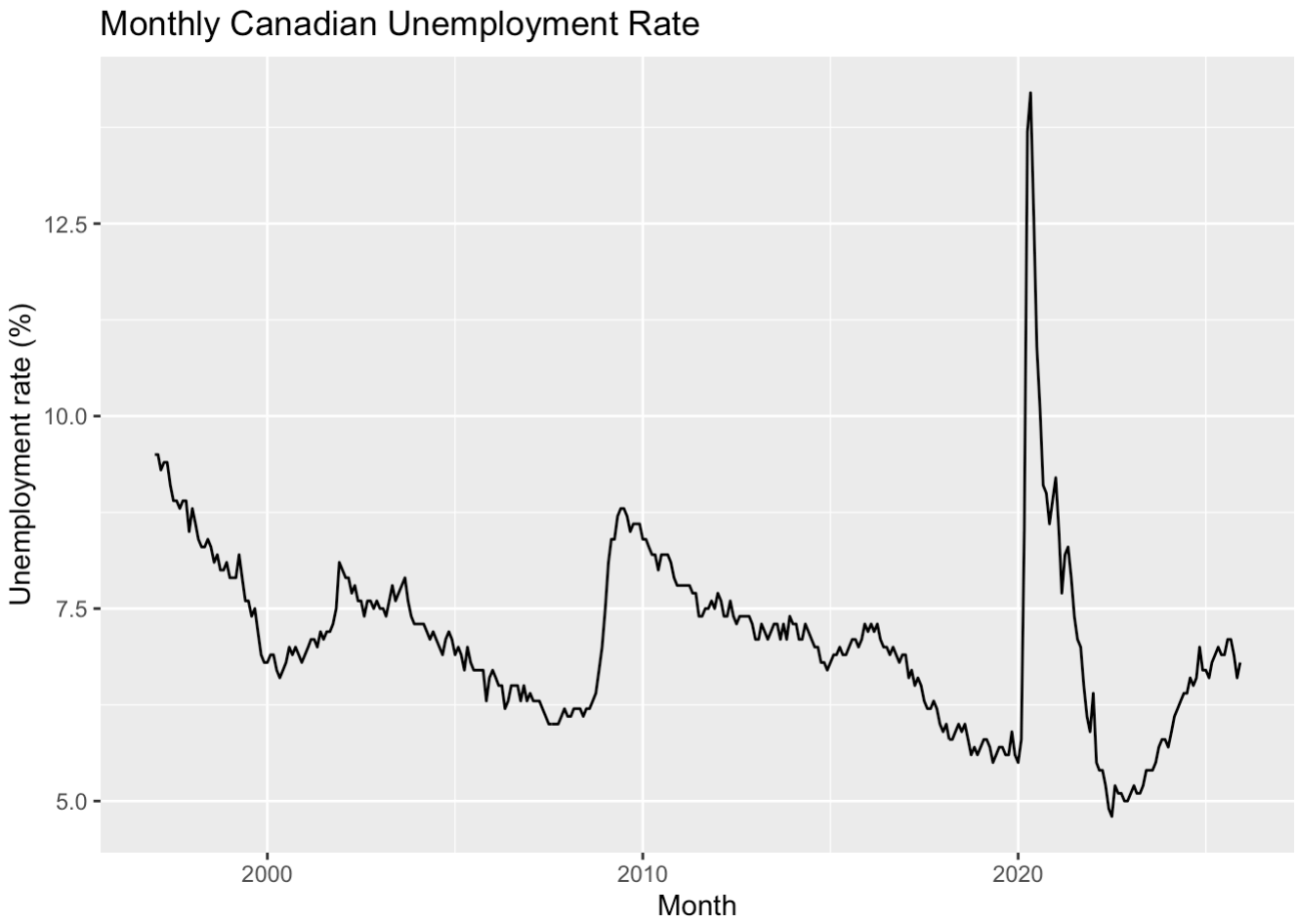
```
##      month            unemployment_rate total_employment    all_gdp
##  Min.   :1997-01-01   Min.   : 4.800    Min.   :13148    Min.   :1200427
##  1st Qu.:2004-03-24   1st Qu.: 6.375    1st Qu.:15794    1st Qu.:1549730
##  Median :2011-06-16   Median : 7.000    Median :17262    Median :1751258
##  Mean   :2011-06-16   Mean   : 7.108    Mean   :17229    Mean   :1782487
##  3rd Qu.:2018-09-08   3rd Qu.: 7.600    3rd Qu.:18546    3rd Qu.:2033314
##  Max.   :2025-12-01   Max.   :14.200    Max.   :21450    Max.   :2339856
```

```
summary_table <- tibble(
  Variable = c("Unemployment rate", "Total employment", "All-industry GDP"),

  Mean = c(
    mean(eda_data$unemployment_rate),
    mean(eda_data$total_employment),
    mean(eda_data$all_gdp)
  ),

  SD = c(
    sd(eda_data$unemployment_rate),
    sd(eda_data$total_employment),
    sd(eda_data$all_gdp)
  ),

  Min = c(
    min(eda_data$unemployment_rate),
    min(eda_data$total_employment),
    min(eda_data$all_gdp)
  ),

  Max = c(
    max(eda_data$unemployment_rate),
    max(eda_data$total_employment),
    max(eda_data$all_gdp)
  )
) %>%
  mutate(across(where(is.numeric), ~ round(.x, 2)))

summary_table
```

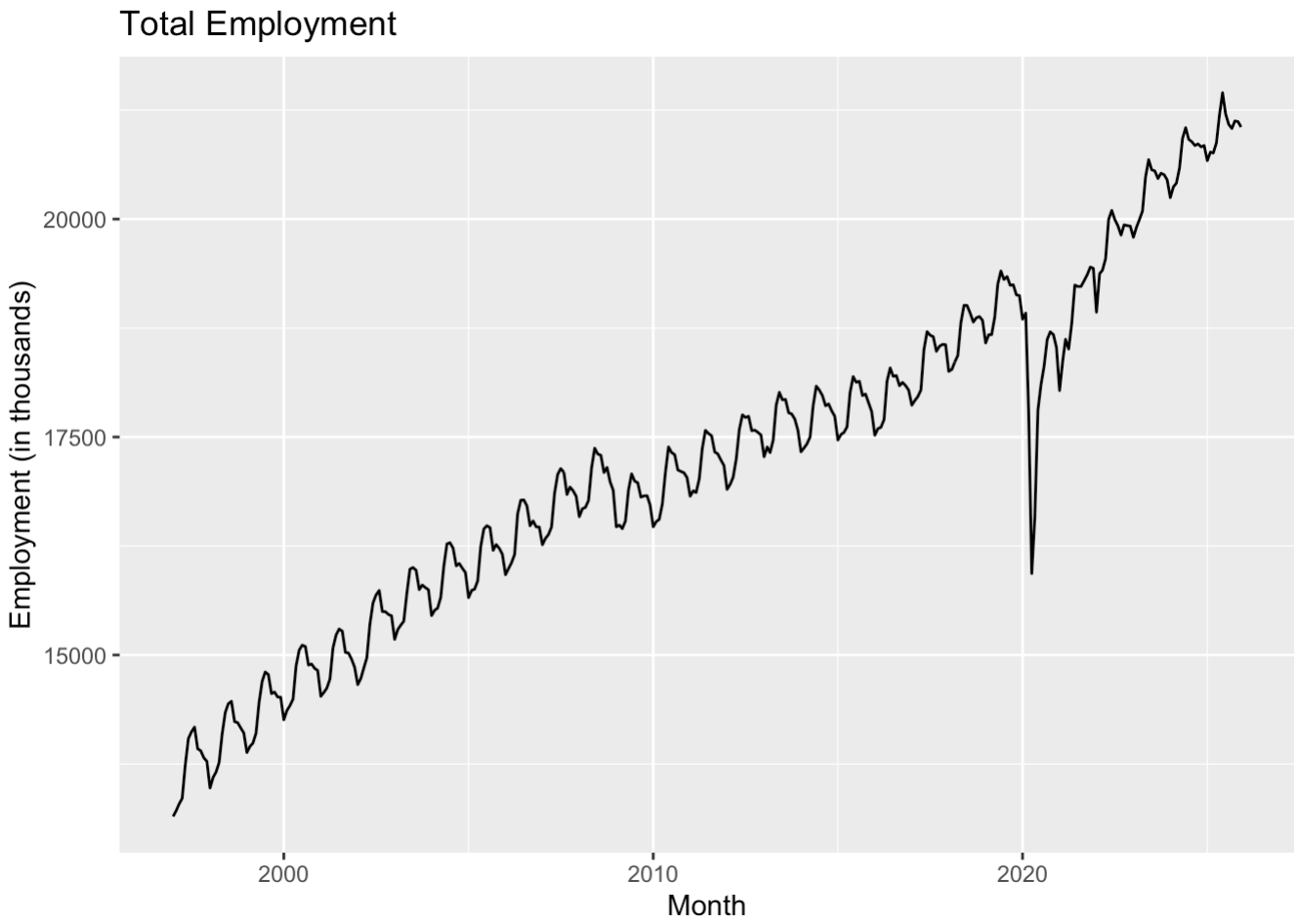
```
## # A tibble: 3 × 5
##   Variable                Mean        SD       Min       Max
##   <chr>                  <dbl>     <dbl>     <dbl>     <dbl>
## 1 Unemployment rate       7.11      1.16       4.8      14.2
## 2 Total employment    17229.     1952.     13148.    21450.
## 3 All-industry GDP  1782487.   306333.   1200427   2339856
```

- Plots

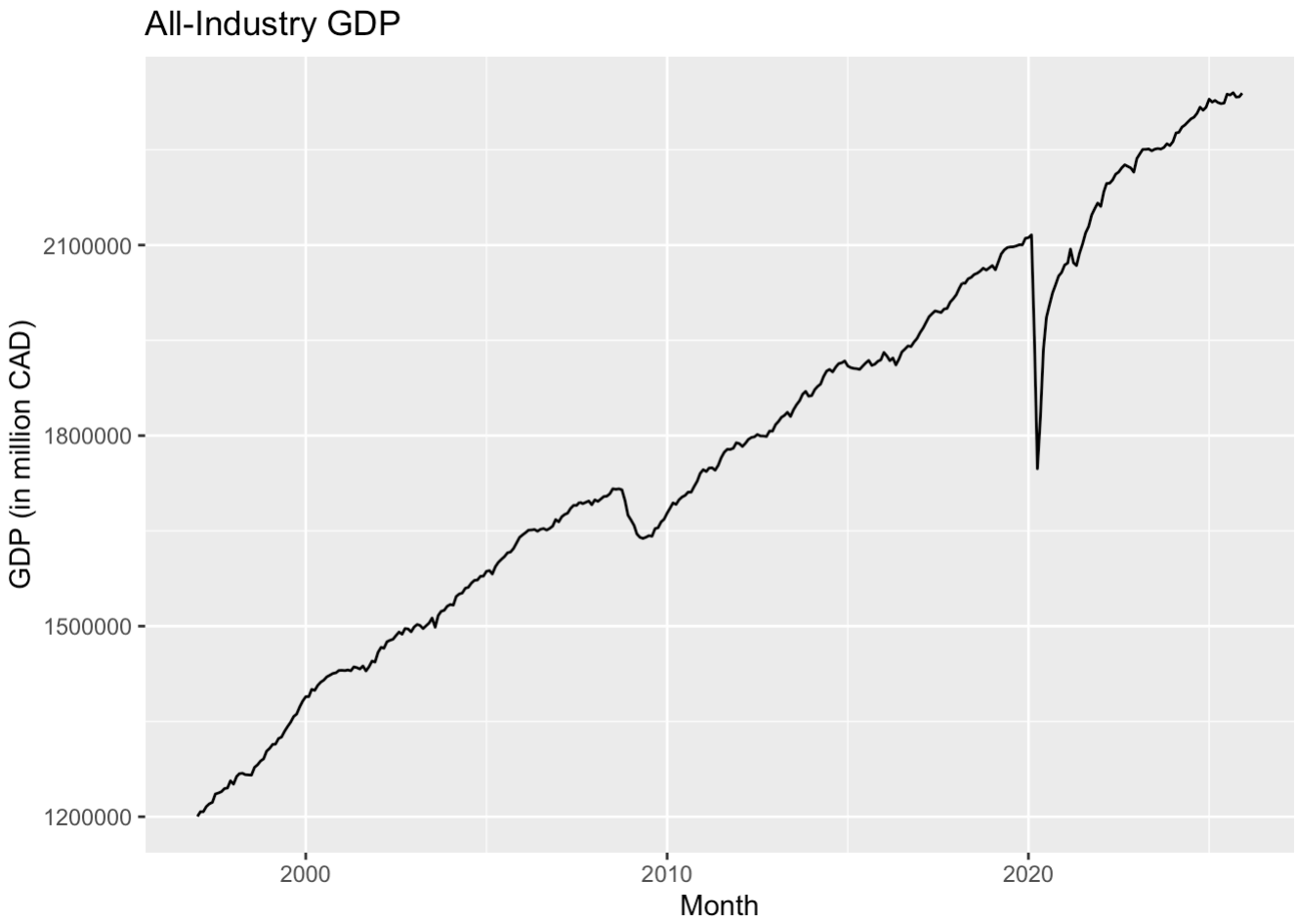
```
# Unemployment Rate plot
options(repr.plot.width = 9, repr.plot.height = 5)
unemp_plot <- ggplot(eda_data, aes(x = month, y = unemployment_rate)) +
    geom_line() +
    labs(
    title = "Monthly Canadian Unemployment Rate",
    x = "Month",
    y = "Unemployment rate (%)")

unemp_plot
```




```
# Total employment plot
options(repr.plot.width = 9, repr.plot.height = 5)
tot_emp_plot <- ggplot(eda_data, aes(x = month, y = total_employment)) +
    geom_line() +
    labs(
    title = "Total Employment ",
    x = "Month",
    y = "Employment (in thousands)")
tot_emp_plot
```

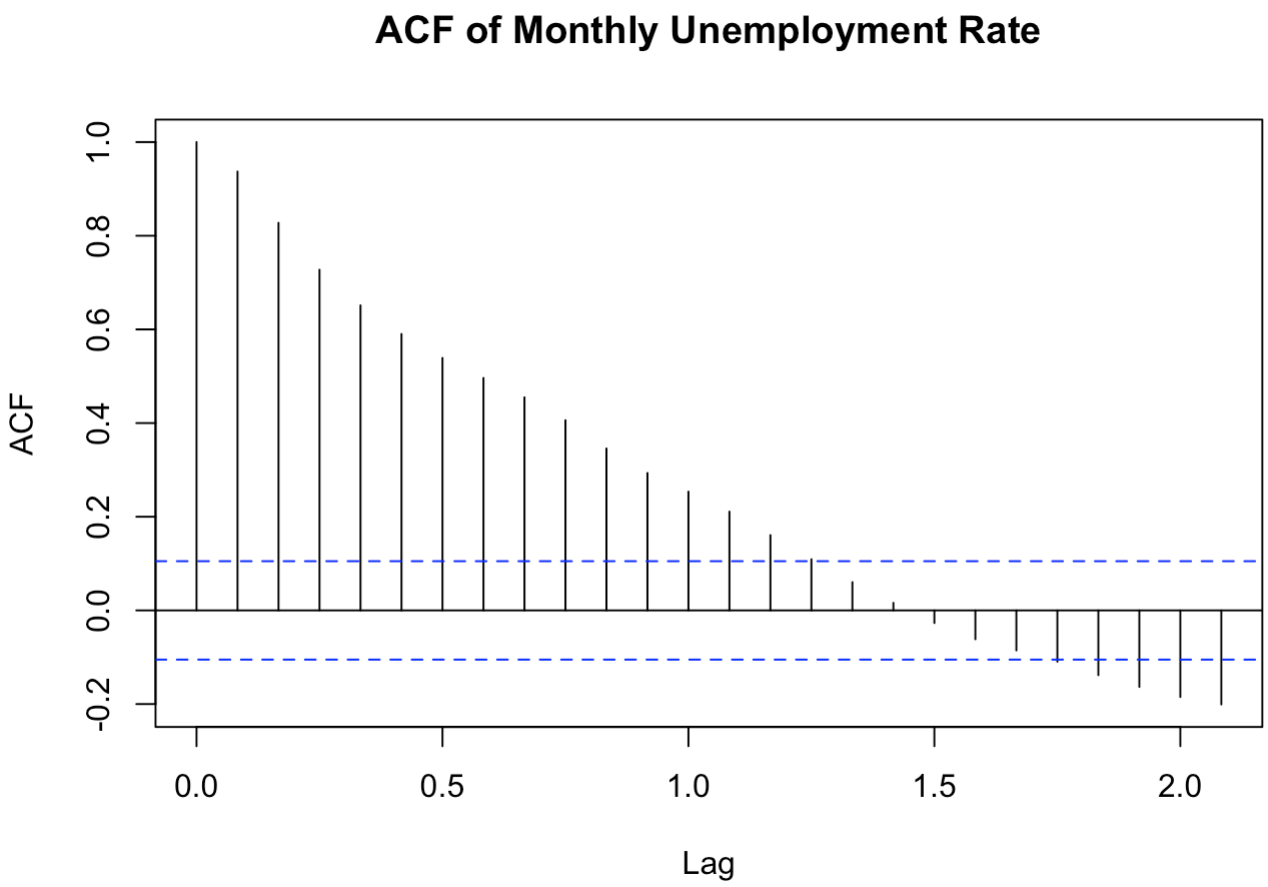



```
# All-industry GDP plot
options(repr.plot.width = 9, repr.plot.height = 5)
all_gdp_plot <- ggplot(eda_data, aes(x = month, y = all_gdp)) +
    geom_line() +
    labs(
    title = "All-Industry GDP",
    x = "Month",
    y = "GDP (in million CAD)")
all_gdp_plot
```

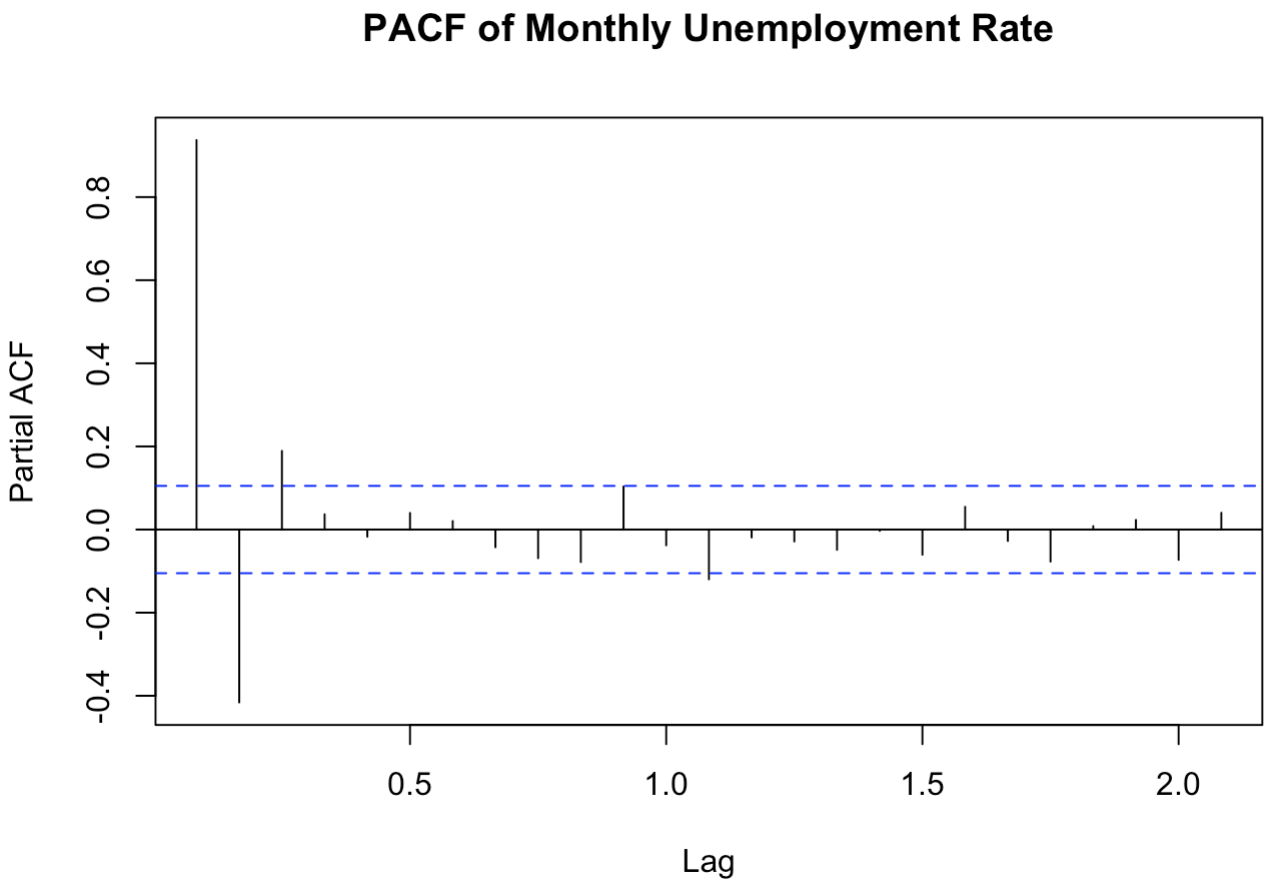


- **ACF and PACF plot of monthly unemployment rate**

```
#Create time series object
unemp_ts <- ts(eda_data$unemployment_rate, start = c(1997, 1), frequency = 12)

#ACF/PACF of the unemployment ts object
acf(unemp_ts, main = "ACF of Monthly Unemployment Rate")
```

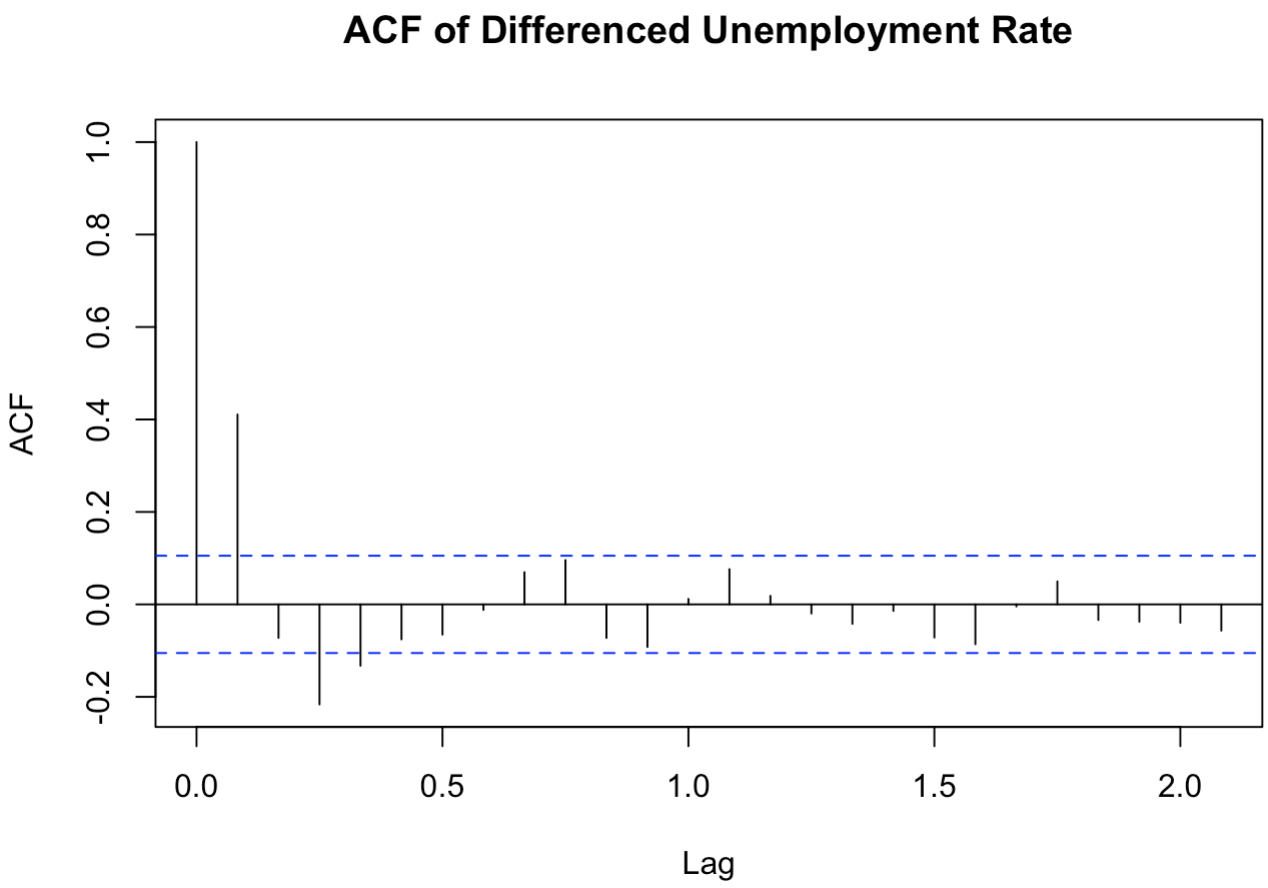



```
pacf(unemp_ts, main = "PACF of Monthly Unemployment Rate")
```

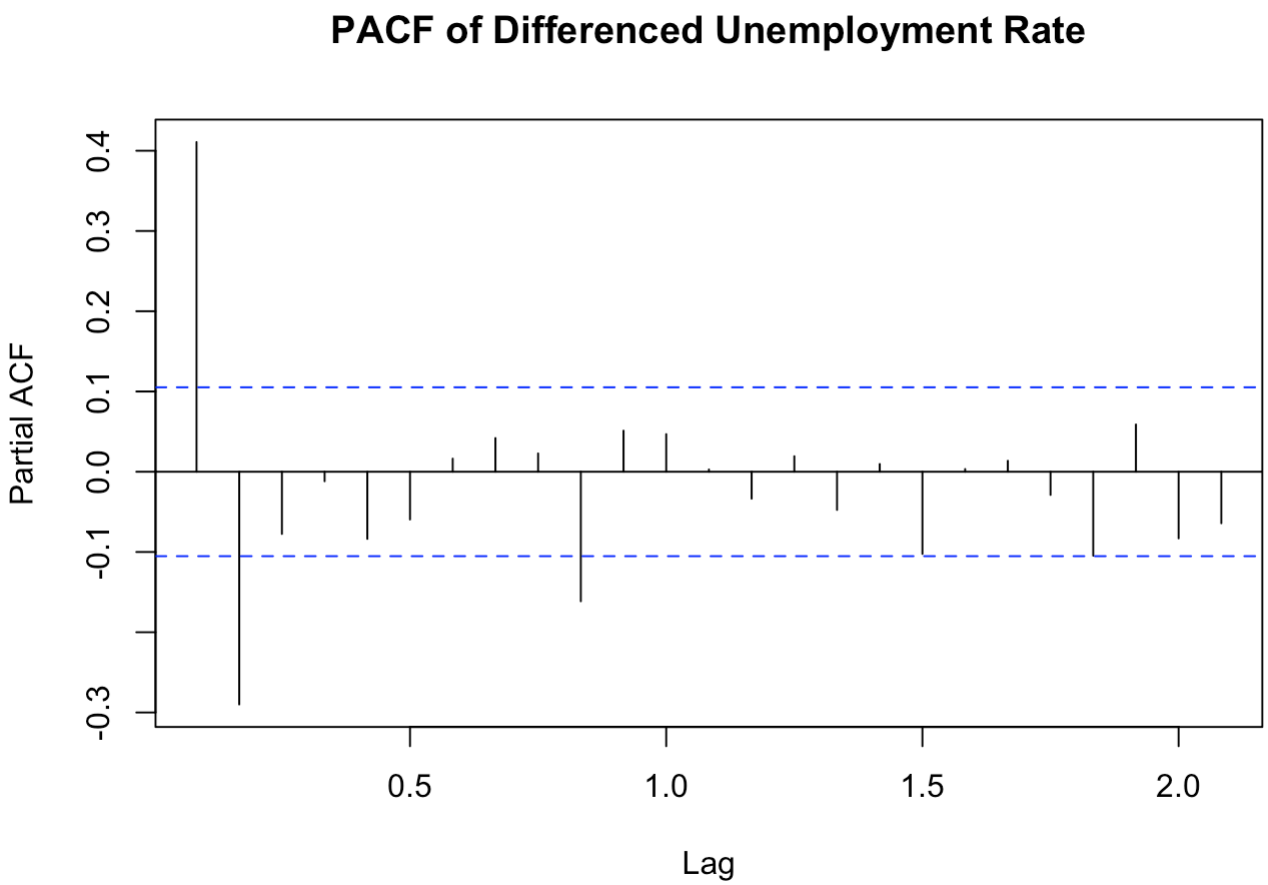


- Monthly unemployment rate is not stationary
- **ACF/PACF of the differenced unemployment ts object**

```
acf(diff(unemp_ts), main = "ACF of Differenced Unemployment Rate")
```




```
pacf(diff(unemp_ts), main = "PACF of Differenced Unemployment Rate")
```



- Find **correlation** between unemployment rate, total_employment and all-industry GDP

```
#Relationship investigation between variables?
library(corrplot)
```

```
## Warning: package 'corrplot' was built under R version 4.3.3
```

```
## corrplot 0.95 loaded
```

```
corr_data <- eda_data %>%
  select(-month)

cor_matrix <- cor(corr_data)
print(cor_matrix)
```

```
##                   unemployment_rate total_employment    all_gdp
## unemployment_rate         1.0000000       -0.4839627 -0.4686860
## total_employment         -0.4839627        1.0000000  0.9869102
## all_gdp                  -0.4686860        0.9869102  1.0000000
```

Step 2.2: Baseline forecast (Persistence and Average)

```
train <- window(unemp_ts, end = c(2018, 12))
test <- window(unemp_ts, start = c(2019, 1))
length(train)
```

```
## [1] 264
```

```
length(test)
```

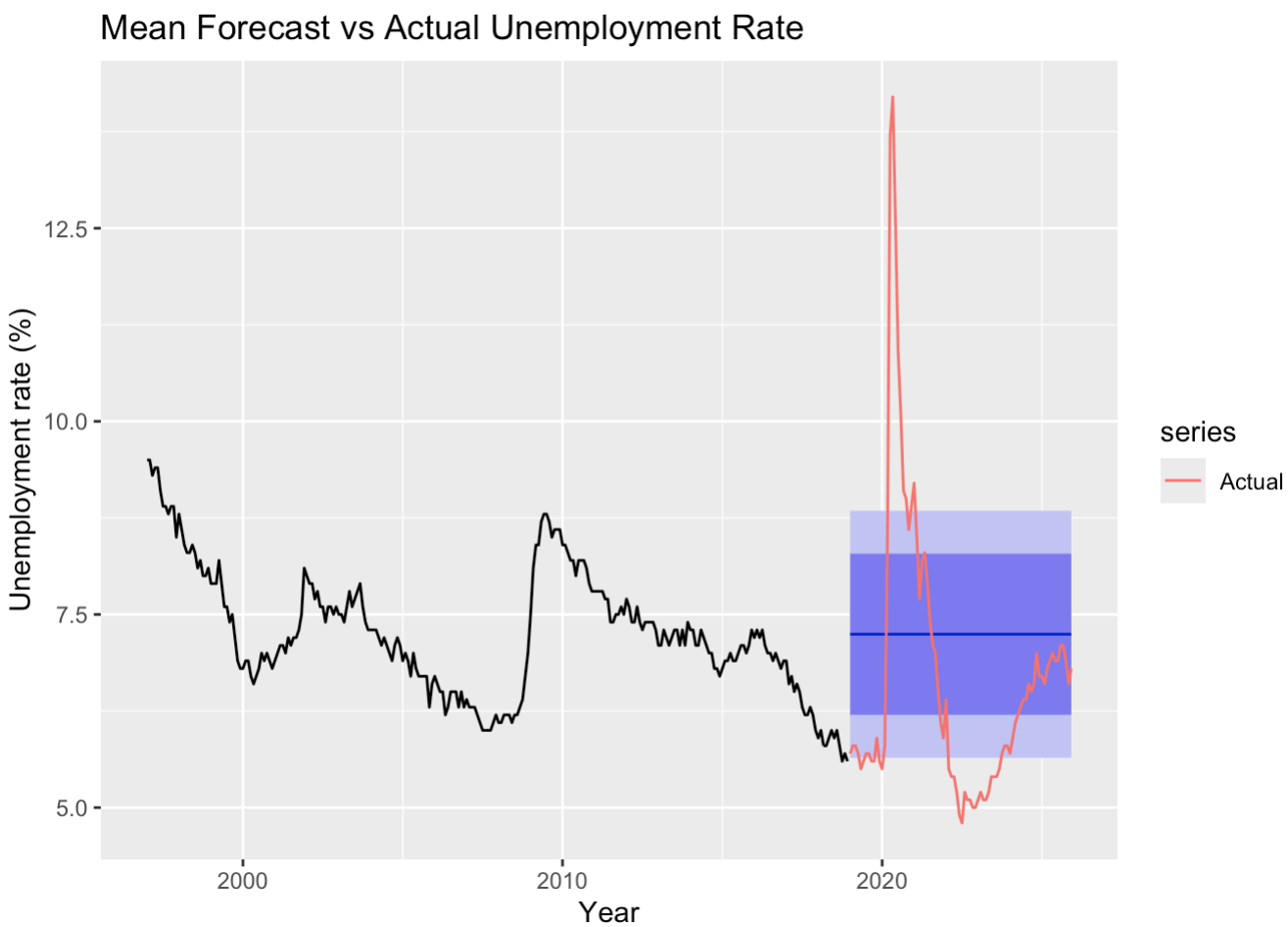
```
## [1] 84
```

- Average forecast

```
mean_fc <- meanf(train, h = length(test))
accuracy(mean_fc, test)
```

```
##                        ME      RMSE       MAE        MPE      MAPE     MASE
## Training set -1.854355e-16 0.8085488 0.6336318  -1.228972  8.805943 1.234920
## Test set     -5.617965e-01 1.9011741 1.5059163 -14.244816 22.965239 2.934964
##                   ACF1 Theil's U
## Training set 0.9593768        NA
## Test set     0.9134465  2.870727
```

```
autoplot(mean_fc) +
  autolayer(test, series = "Actual") +
  labs(
    title = "Mean Forecast vs Actual Unemployment Rate",
    x = "Year",
    y = "Unemployment rate (%)"
  )
```

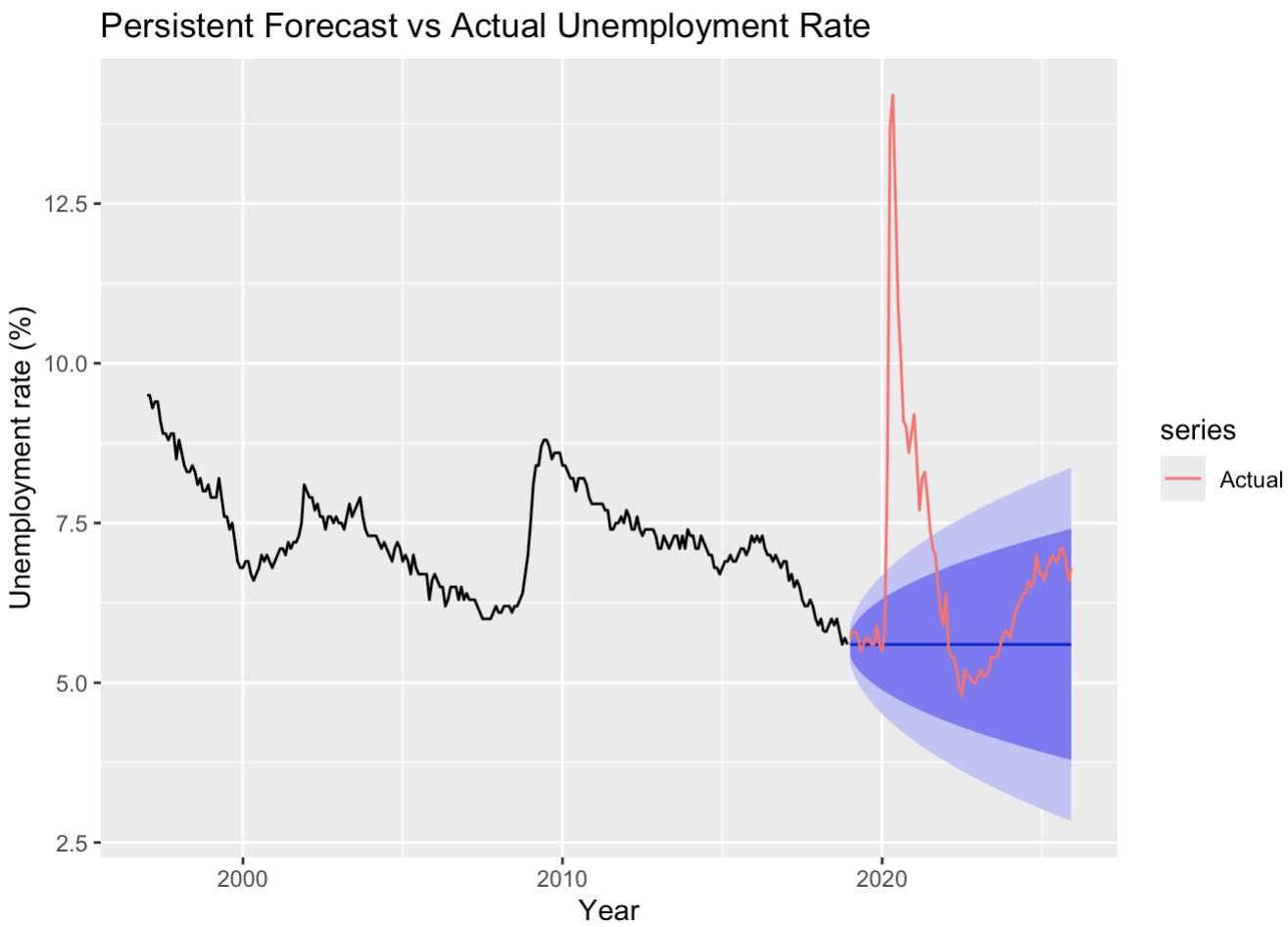


- Persistence forecast

```
persistent_fc <- naive(train, h = length(test))
accuracy(persistent_fc, test)
```

```
##                       ME      RMSE       MAE        MPE     MAPE      MASE
## Training set -0.0148289 0.1539098 0.1159696 -0.2232677  1.60905 0.2260196
## Test set      1.0821429 2.1142093 1.2773810 11.6818989 15.50410 2.4895592
##                     ACF1 Theil's U
## Training set 0.005416297        NA
## Test set     0.913446520  2.370618
```

```
autoplot(persistent_fc) +
  autolayer(test, series = "Actual") +
  labs(
    title = "Persistent Forecast vs Actual Unemployment Rate",
    x = "Year",
    y = "Unemployment rate (%)"
  )
```



Step 3.1: Holt exponential smoothing

```
# install.packages("forecast")
library(forecast)
```

```
holt_fc <- holt(train, h = length(test))
plot(holt_fc,  main = "Forecasts from Holt's method", xlab = "Year", ylab = "Unemploy
ment rate (%)", sub = "Figure 8. Holt Forecast vs Actual value on Test Set")
lines(test, col = "red")
```

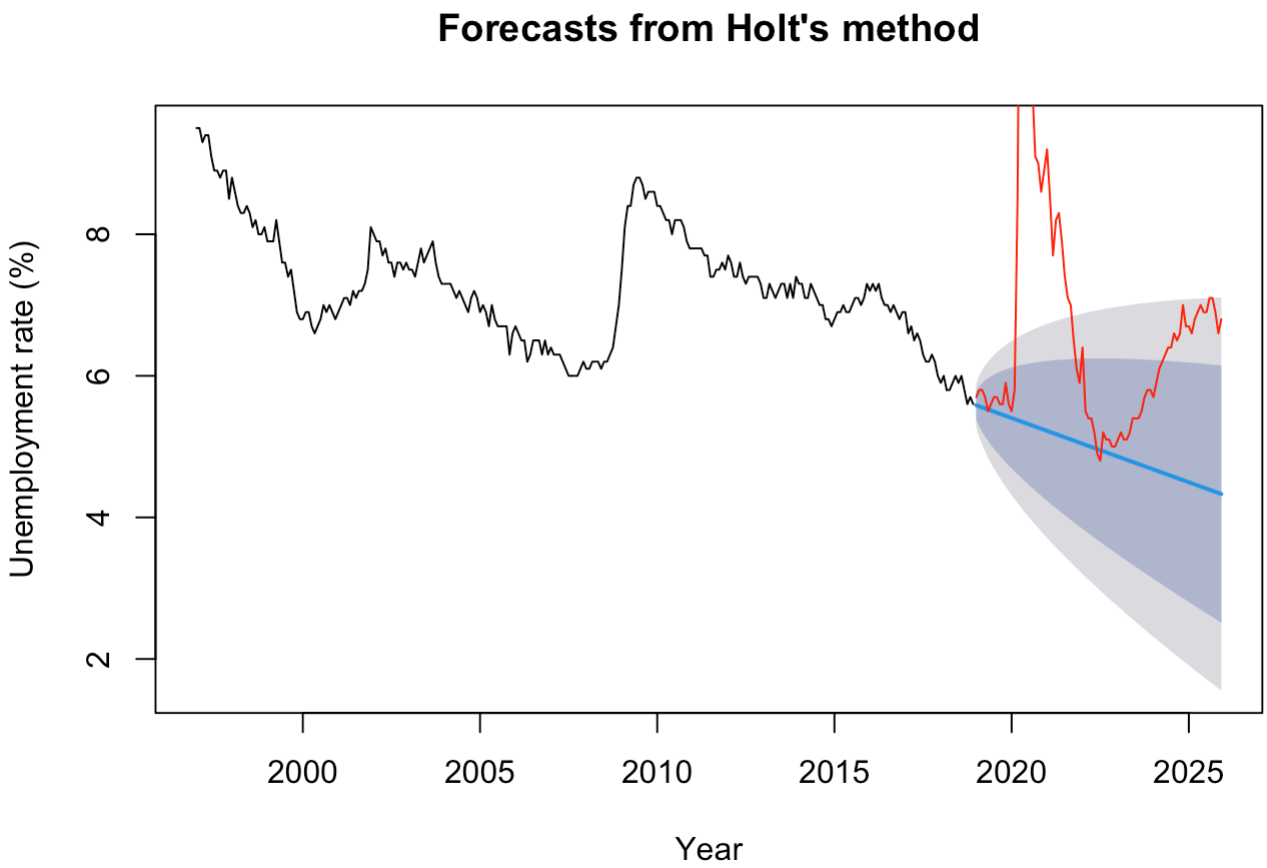


Figure 8. Holt Forecast vs Actual value on Test Set

```
accuracy(holt_fc, test)
```

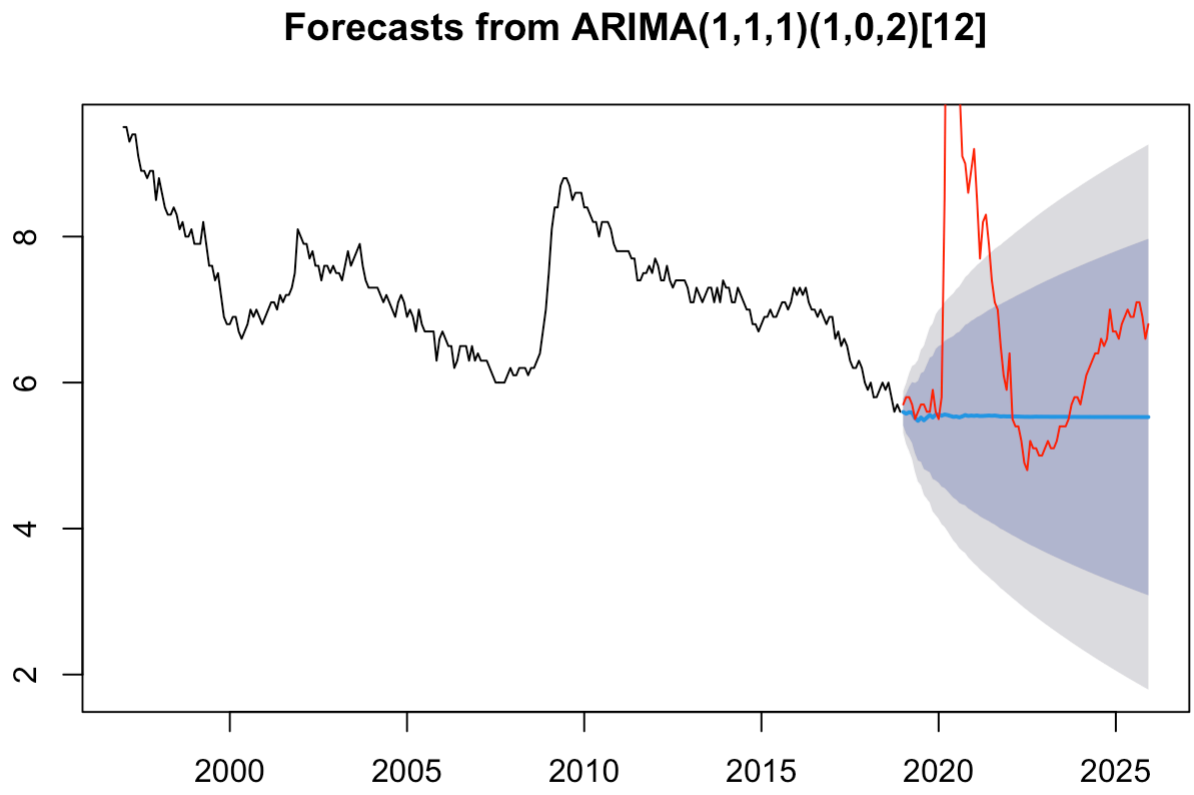
```
##                        ME      RMSE       MAE         MPE      MAPE     MASE
## Training set 0.0002462728 0.1529177 0.1179421 -0.01206121  1.634964 0.229864
## Test set     1.7245054526 2.4753805 1.7302154 21.94592184 22.062670 3.372114
##                     ACF1 Theil's U
## Training set 0.007036993        NA
## Test set     0.905270168   3.02168
```

- From above, we observe that the training error is low, indicating a good fit, however, the test RMSE is significantly higher (2.48), thus showing poor forecasting performance.

  This is mainly due to the model's inability to capture the sudden spike in unemployment during COVID-19 in 2020. Additionally, the high residual autocorrelation (ACF1 = 0.905) suggests that the model does not fully capture the underlying time dependence in the series.

Step 3.2: ARIMA model

```
#Training set
fit_auto <- auto.arima(train)

#test set
arima_fc <- forecast(fit_auto, h=length(test))
plot(arima_fc)
lines(test, col="red")
```




```
accuracy(arima_fc, test)
```

```
##                       ME      RMSE       MAE        MPE     MAPE      MASE
## Training set -0.01028619 0.1427994 0.1108049 -0.1519841  1.52937 0.2159538
## Test set      1.14644262 2.1457760 1.3063259 12.7037785 15.84813 2.5459716
##                     ACF1 Theil's U
## Training set -0.08611533        NA
## Test set      0.91294604  2.411667
```

- Based on the RMSE, ARIMA performs slightly better than Holt's model since it has lower RMSE. However, it still struggles to capture the sudden spike during COVID, leading to large prediction errors.

  The forecasts are relatively flat and fail to adapt to abrupt changes in the data.

  Thus, univariate models may be insufficient motivating the use of ARIMAX models with external economic indicators.

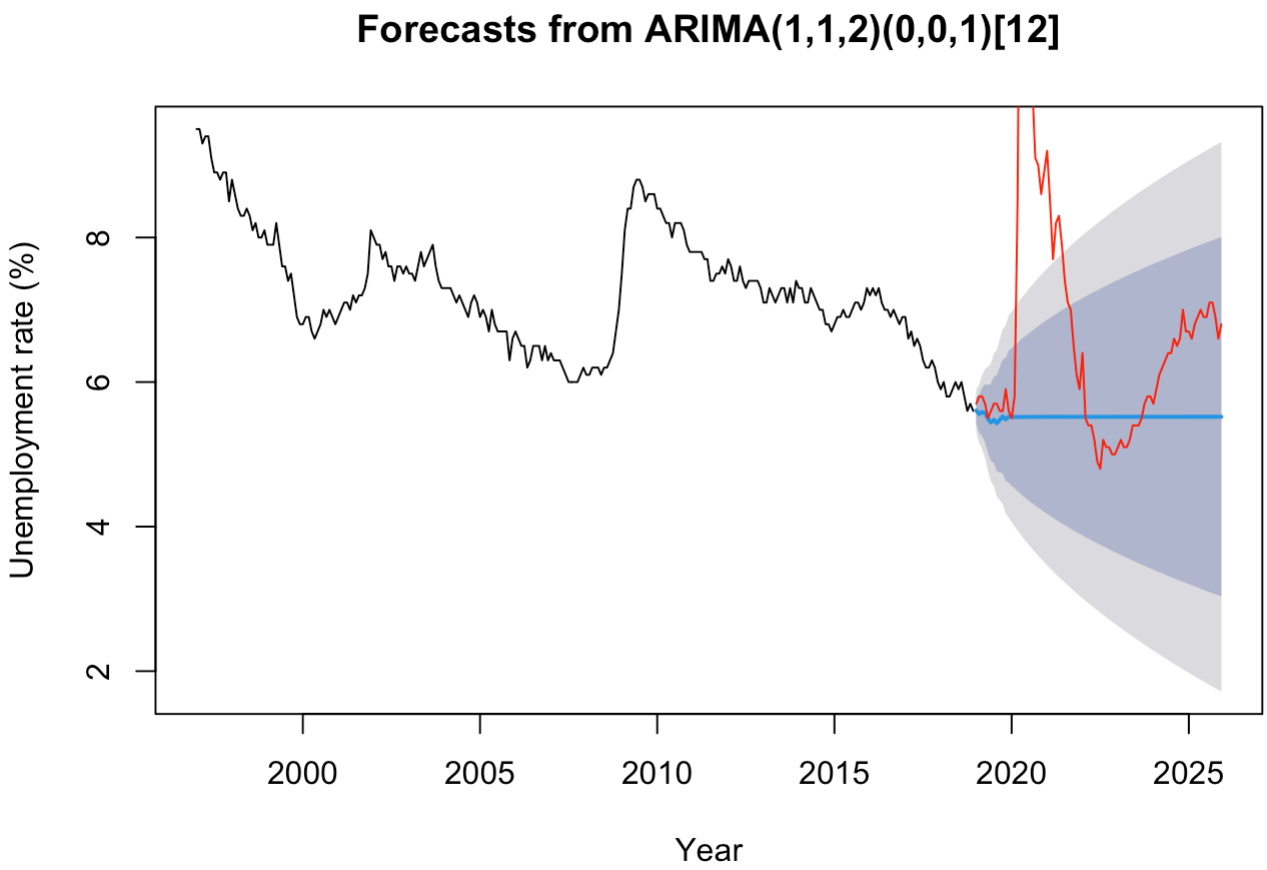
- Trying a few more models - we need differencing due to non staionarity

```
fit1 <- Arima(train, order=c(1,1,1))
fit2 <- Arima(train, order=c(2,1,1))
fit3 <- Arima(train, order=c(1,1,2))
# adding extra models as suggested in EDA
fit4 <- Arima(train, order=c(0,1,1))
fit5 <- Arima(train, order=c(1,1,0))


AIC(fit_auto, fit1, fit2, fit3, fit4, fit5)
```

```
##          df       AIC
## fit_auto  6 -262.3435
## fit1      3 -237.2648
## fit2      4 -237.6317
## fit3      4 -238.0763
## fit4      2 -234.0363
## fit5      2 -234.0436
```

- We observe that the model selected by auto.arima outperforms all the manual ARIMA models fitted above, as it has the lowest AIC (-262.34)

```
# trying for another model using
# >auto.arima(train, stepwise=FALSE, approximation=FALSE)
# gives us ARIMA(1,1,2)(0,0,1)[12] with AIC = -264.88

fit_new <- Arima(train, order=c(1,1,2), seasonal=c(0,0,1))
arima_fc_new <- forecast(fit_new, h=length(test))
plot(arima_fc_new,
     main = "Forecasts from ARIMA(1,1,2)(0,0,1)[12]",
     xlab = "Year",
     ylab = "Unemployment rate (%)")
lines(test, col="red")
```




```
accuracy(arima_fc_new, test)
```

```
##                       ME      RMSE       MAE        MPE      MAPE      MASE
## Training set -0.01015301 0.1426689 0.1113921 -0.1493692  1.538004 0.2170983
## Test set      1.16381953 2.1571472 1.3163273 12.9710162 15.975012 2.5654638
##                      ACF1 Theil's U
## Training set -0.004710084        NA
## Test set      0.913020315  2.427475
```

```
AIC(fit_auto,fit_new)
```

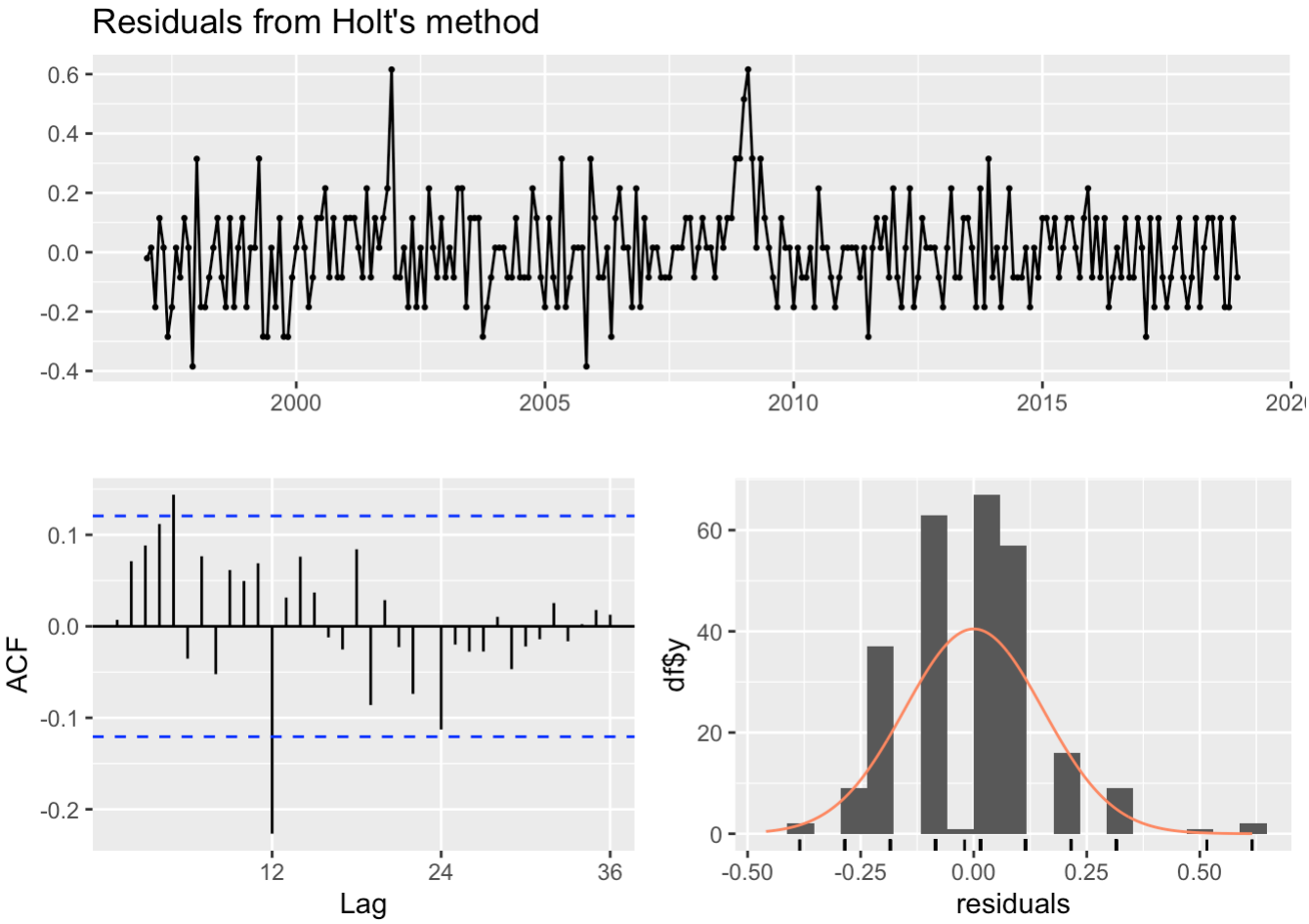
```
##          df       AIC
## fit_auto  6 -262.3435
## fit_new   5 -264.8781
```

- By comparing the ARIMA (1,1,1)(1,0,2)[12] and ARIMA(1,1,2)(0,0,1)[12], we observe that first has a higher AIC(-262.34) but lower RMSE(2.14),whereas the latter has a lower AIC (-264.88) and higher RMSE (2.15)

  Thus, the new ARIMA model fits the training data better as indicated by the low AIC, but does not improve prediction accuracy, indicated by higher RMSE. Thus, this suggests overfitting and hence we consider ARIMA (1,1,1)(1,0,2)[12] as our final ARIMA model.

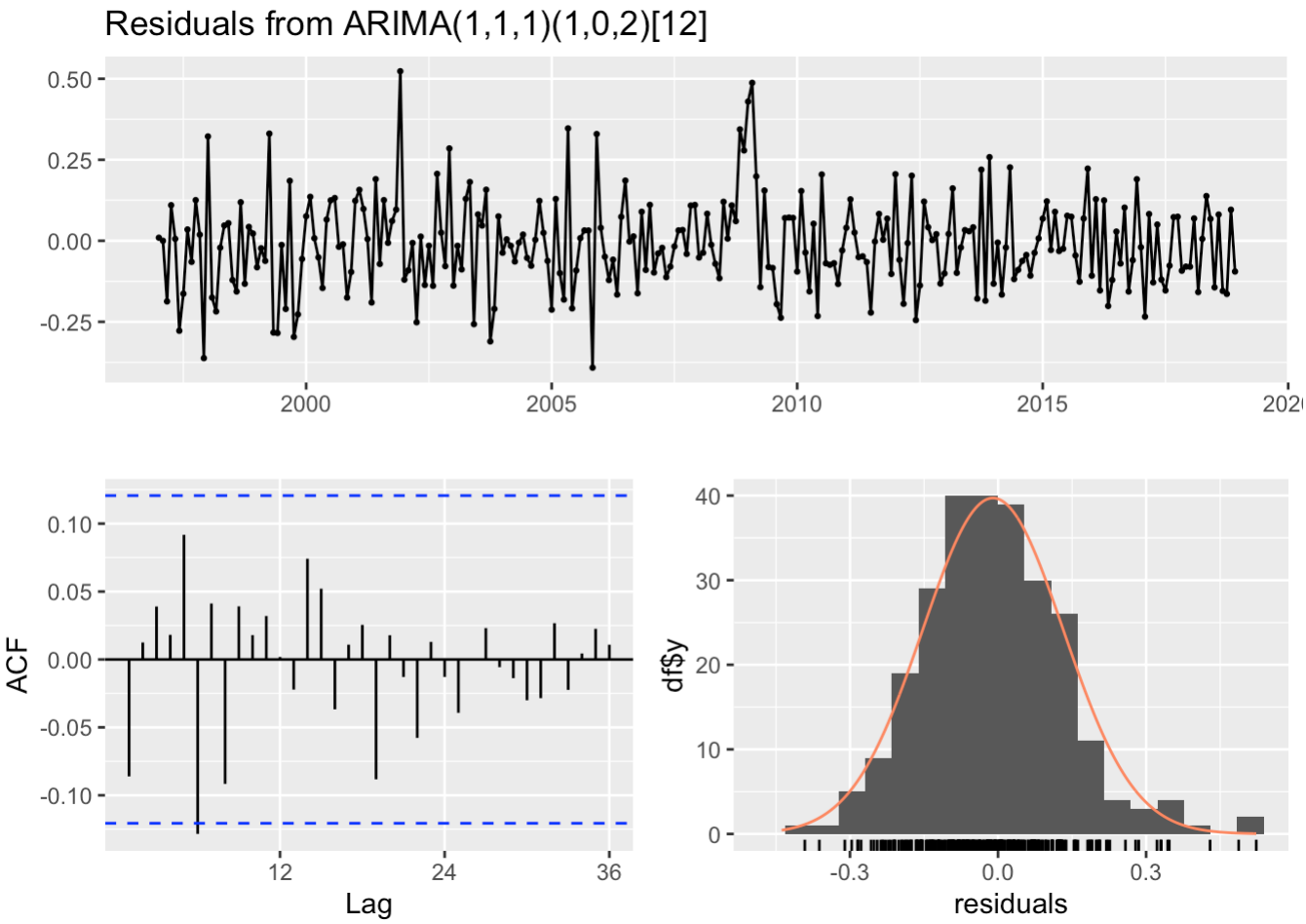
  The original series exhibits non-stationarity due to trend, which is also addressed through first-order differencing (d = 1) in the ARIMA models.

```
checkresiduals(holt_fc)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from Holt's method
## Q* = 44.886, df = 24, p-value = 0.006008
##
## Model df: 0.   Total lags used: 24
```

```
checkresiduals(fit_auto)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1)(1,0,2)[12]
## Q* = 19.344, df = 19, p-value = 0.435
##
## Model df: 5.   Total lags used: 24
```

```
Box.test(residuals(fit_auto), lag=20, type="Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  residuals(fit_auto)
## X-squared = 18.231, df = 20, p-value = 0.5722
```

- The residual diagnostics for Holt's method show significant autocorrelation, indicating that the residuals are not random, and thus, says that the model fails to capture temporal patterns in the data. (Since the p-value is also 0.006 < 0.05, this also implies that <u>we reject the hypothesis that the residuals are random</u>)

  The ARIMA model produces residuals closer to white noise, with most autocorrelations lying within the confidence bounds, thus, indicating that the model captures underlying temporal dependence better than Holt's method. Here the p-value = 0.435 > 0.05 => <u>we do not reject that the residuals are random</u>

  Similarly in the box test, we observe a p-value = 0.5722 > 0.05, thus we arrive at the same conclusion of <u>not rejecting for the ARIMA model</u>

Step 3.3: ARIMAX model

- Load data
    - unemp_ts <- ts(eda_data$unemployment_rate, start = c(1997, 1), frequency = 12)
    - train <- window(unemp_ts, end = c(2018, 12))
    - test <- window(unemp_ts, start = c(2019, 1))
- Transform GDP into log(gdp) since it's an economic indicator

```
loggdp <- log(eda_data$all_gdp)
employ <- eda_data$total_employment
```

- Find the correlation between two predictors

```
cor(loggdp,employ)
```

```
## [1] 0.9821454
```

- Noticing a high correlation rate between log_gdp and total_employment, we can choose to ignore total_employment to train models faster.
- Create time series for log(gdp) , and split it into test and train data

```
xreg_ts <- ts(loggdp, start = c(1997, 1), frequency = 12)

xreg_train <- window(xreg_ts, end=c(2018,12))
xreg_test  <- window(xreg_ts, start=c(2019,1))

length(train)
```

```
## [1] 264
```

```
length(test)
```

```
## [1] 84
```

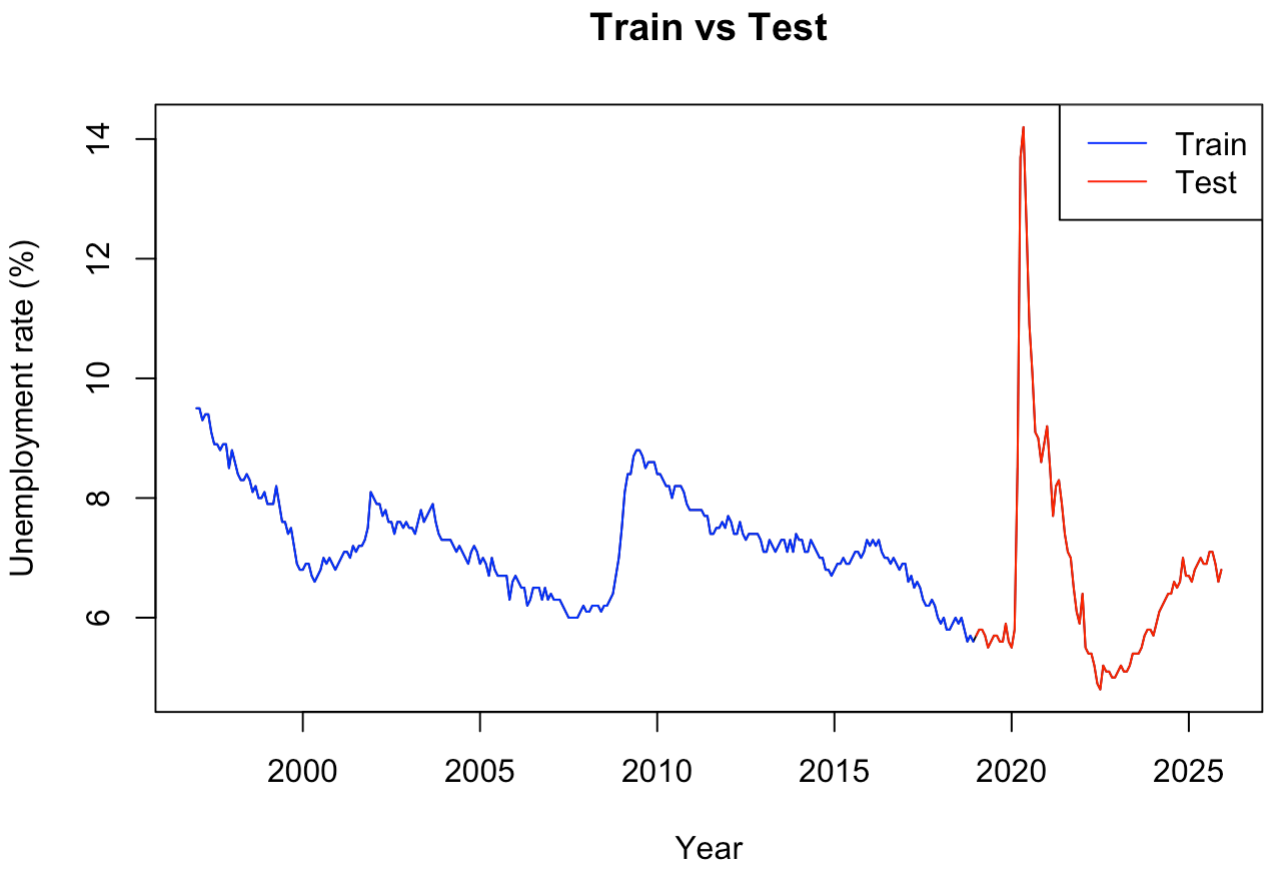
```
length(xreg_train)
```

```
## [1] 264
```

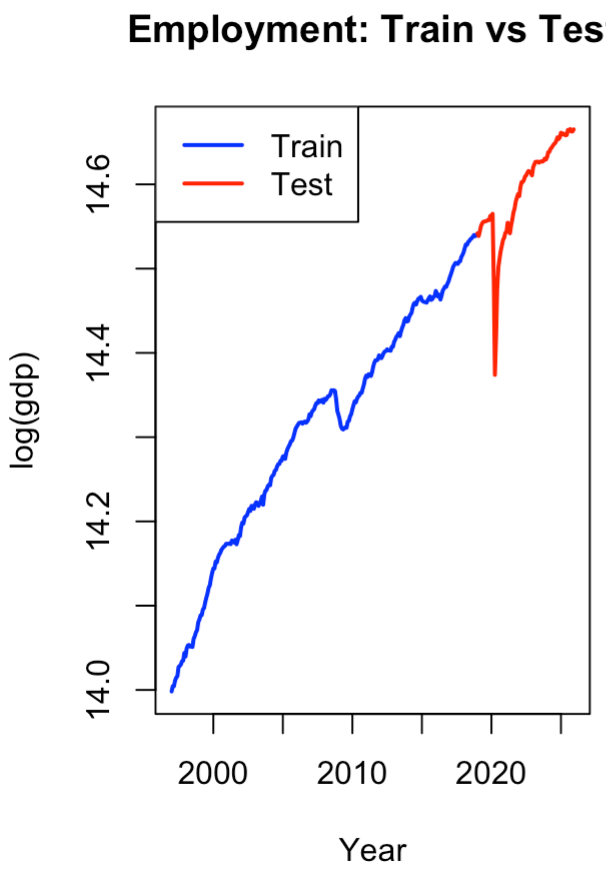
```
length(xreg_test)
```

```
## [1] 84
```

```
plot(unemp_ts,
     col = "black",
     main = "Train vs Test",
     xlab = "Year",
     ylab = "Unemployment rate (%)")

lines(train, col = "blue")
lines(test, col = "red")

legend("topright",
       legend = c("Train", "Test"),
       col = c("blue", "red"),
       lty = 1)
```



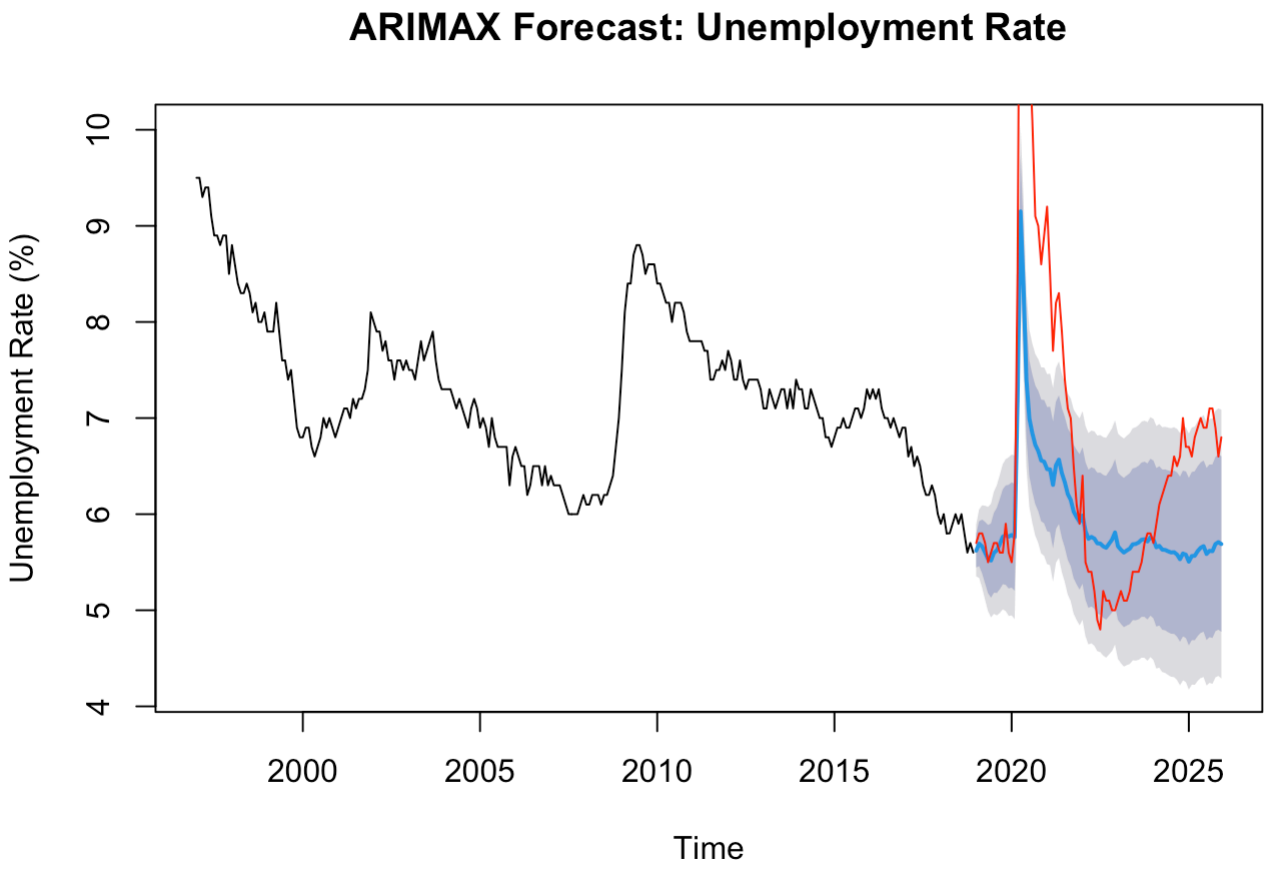

```
par(mfrow = c(1, 2))

plot(xreg_ts, col = "black", main = "Employment: Train vs Test", ylab = "log(gdp)", x
lab="Year")
lines(xreg_train, col = "blue", lwd = 2)
lines(xreg_test, col = "red", lwd = 2)
legend("topleft", legend = c("Train", "Test"), col = c("blue", "red"), lty = 1, lwd =
2)
```




```
armax_model_auto  <- auto.arima(train, xreg = xreg_train,
                           seasonal = TRUE,
                           stepwise = FALSE,
                           approximation = FALSE)

summary(armax_model_auto)
```

```
## Series: train
## Regression with ARIMA(0,1,1)(0,0,2)[12] errors
##
## Coefficients:
##           ma1     sma1     sma2   drift      xreg
##       -0.1197  -0.3453  -0.1070  0.0230  -17.3906
## s.e.   0.0649   0.0640   0.0609  0.0065    2.4895
##
## sigma^2 = 0.01855:  log likelihood = 154.33
## AIC=-296.67   AICc=-296.34   BIC=-275.24
##
## Training set error measures:
##                        ME      RMSE       MAE         MPE     MAPE      MASE
## Training set -0.0002100046 0.1346365 0.1047598 -0.04371199 1.442263 0.2041722
##                     ACF1
## Training set 0.003474615
```

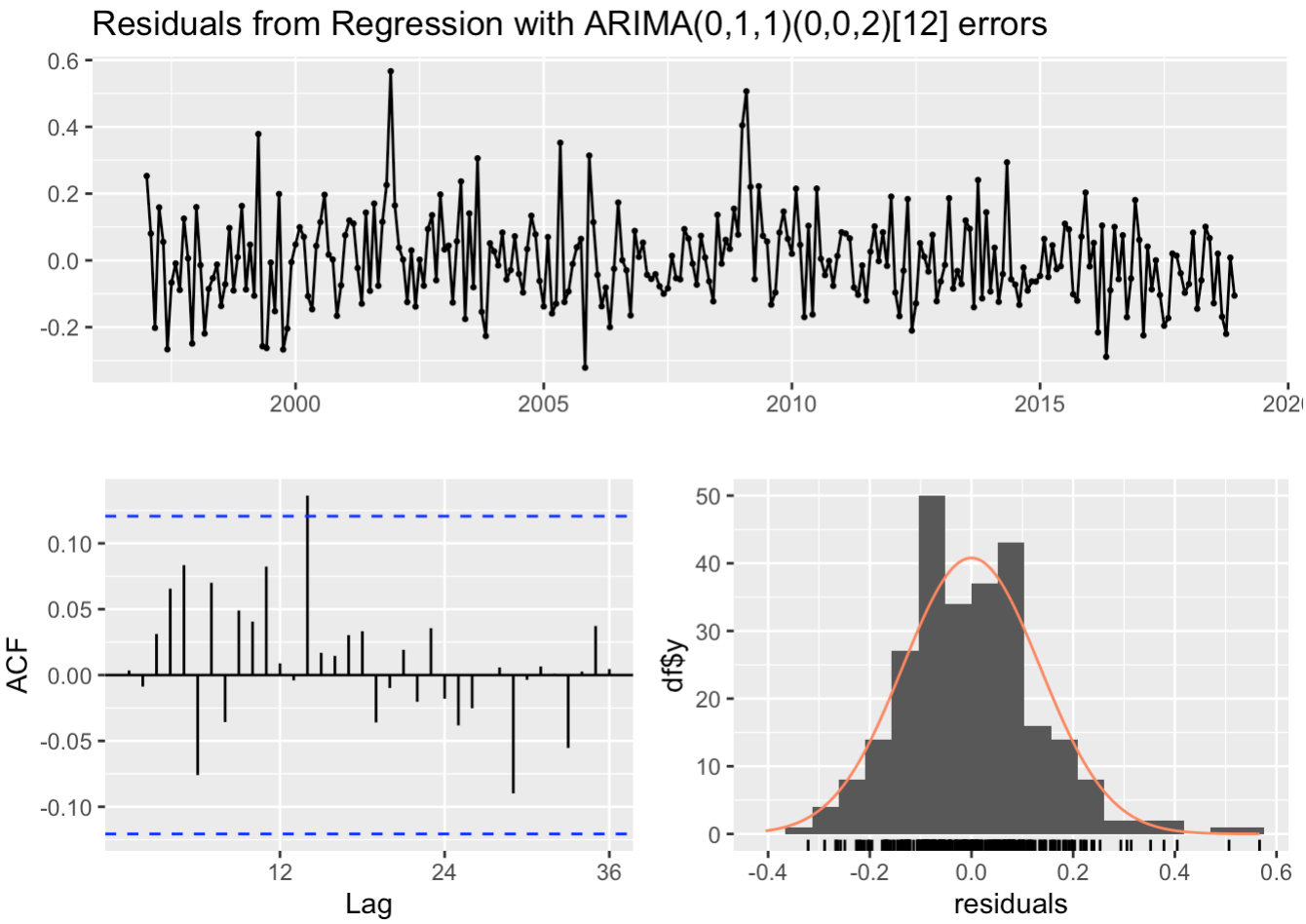
```
armax_fc_auto <- forecast(armax_model_auto, xreg = xreg_test)
plot(armax_fc_auto,
     main = "ARIMAX Forecast: Unemployment Rate",
     ylab = "Unemployment Rate (%)",
     xlab = "Time")

lines(test, col = "red")
```




```
accuracy(armax_fc_auto, test)
```

```
##                        ME      RMSE       MAE         MPE      MAPE      MASE
## Training set -0.0002100046 0.1346365 0.1047598 -0.04371199  1.442263 0.2041722
## Test set      0.7386363216 1.4952357 0.9964941  7.72621210 12.727591 1.9421231
##                     ACF1 Theil's U
## Training set 0.003474615        NA
## Test set     0.923733039  1.732153
```

```
checkresiduals(armax_fc_auto)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from Regression with ARIMA(0,1,1)(0,0,2)[12] errors
## Q* = 16.643, df = 21, p-value = 0.7325
##
## Model df: 3.   Total lags used: 24
```

```
Box.test(residuals(armax_model_auto), lag = 20, type = "Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  residuals(armax_model_auto)
## X-squared = 15.953, df = 20, p-value = 0.7195
```

- Since our p-value for the auto fitted ARMAX model is 0.7325 > 0.05, show our model has captured most mathematical patterns in the data. This is also apparent as our residuals are mostly random noise (except for a few large spikes).

  Like this ARIMA model, the ARMAX model too failed to perform well for the large spike during 2020, but the 95% confidence interval usually captures the true value of unemployment percentage.

- More models with slight variation to see if they perform better on out of sample data.

```
model_1 <- Arima(train, xreg = xreg_train, order = c(0, 1, 1), seasonal = c(0, 0, 1),
include.drift = TRUE)
model_2 <- Arima(train, xreg = xreg_train, order = c(0, 1, 0), seasonal = c(0, 0, 1),
include.drift = TRUE)
model_3 <- Arima(train, xreg = xreg_train, order = c(1, 1, 0), seasonal = c(0, 0, 1),
include.drift = TRUE)
model_4 <- Arima(train, xreg = xreg_train, order = c(1, 1, 1), seasonal = c(0, 0, 1),
include.drift = TRUE)
model_5 <- Arima(train, xreg = xreg_train, order = c(0, 1, 2), seasonal = c(0, 0, 1),
include.drift = TRUE)

model_6  <- Arima(train, xreg = xreg_train, order = c(0, 1, 1), seasonal = c(0, 1,
1))
model_7  <- Arima(train, xreg = xreg_train, order = c(0, 1, 0), seasonal = c(0, 1,
1))
model_8  <- Arima(train, xreg = xreg_train, order = c(1, 1, 0), seasonal = c(0, 1,
1))
model_9  <- Arima(train, xreg = xreg_train, order = c(1, 1, 1), seasonal = c(0, 1,
1))
model_10 <- Arima(train, xreg = xreg_train, order = c(0, 1, 2), seasonal = c(0, 1,
1))
# model_11 is the original baseline model
model_11 <- Arima(train, xreg = xreg_train, order = c(0, 1, 1), seasonal = c(0, 0,
2), include.drift = TRUE)
model_12 <- Arima(train, xreg = xreg_train, order = c(0, 1, 0), seasonal = c(0, 0,
2), include.drift = TRUE)
model_13 <- Arima(train, xreg = xreg_train, order = c(1, 1, 0), seasonal = c(0, 0,
2), include.drift = TRUE)
model_14 <- Arima(train, xreg = xreg_train, order = c(1, 1, 1), seasonal = c(0, 0,
2), include.drift = TRUE)
model_15 <- Arima(train, xreg = xreg_train, order = c(0, 1, 2), seasonal = c(0, 0,
2), include.drift = TRUE)

armax_fc_1  <- forecast(model_1,  xreg = xreg_test)
armax_fc_2  <- forecast(model_2,  xreg = xreg_test)
armax_fc_3  <- forecast(model_3,  xreg = xreg_test)
armax_fc_4  <- forecast(model_4,  xreg = xreg_test)
armax_fc_5  <- forecast(model_5,  xreg = xreg_test)

armax_fc_6  <- forecast(model_6,  xreg = xreg_test)
armax_fc_7  <- forecast(model_7,  xreg = xreg_test)
armax_fc_8  <- forecast(model_8,  xreg = xreg_test)
armax_fc_9  <- forecast(model_9,  xreg = xreg_test)
armax_fc_10 <- forecast(model_10, xreg = xreg_test)

armax_fc_11 <- forecast(model_11, xreg = xreg_test)
armax_fc_12 <- forecast(model_12, xreg = xreg_test)
armax_fc_13 <- forecast(model_13, xreg = xreg_test)
armax_fc_14 <- forecast(model_14, xreg = xreg_test)
armax_fc_15 <- forecast(model_15, xreg = xreg_test)

accuracy(armax_fc_1,  test)
```

```
##                        ME      RMSE      MAE         MPE      MAPE      MASE
## Training set 4.285926e-05 0.1354542 0.105718 -0.03983024  1.456106 0.2060397
## Test set     8.574571e-01 1.5555810 1.041711  9.60503263 13.201507 2.0302496
##                     ACF1 Theil's U
## Training set 0.002092843        NA
## Test set     0.922536527  1.811424
```

```
accuracy(armax_fc_2,  test)
```

```
##                         ME      RMSE       MAE        MPE      MAPE      MASE
## Training set -5.981716e-05 0.1360483 0.1059936 -0.0372763  1.459689 0.2065769
## Test set      9.054060e-01 1.6126921 1.0751684 10.2524120 13.571872 2.0954557
##                     ACF1 Theil's U
## Training set -0.09568596        NA
## Test set      0.92124722   1.87717
```

```
accuracy(armax_fc_3,  test)
```

```
##                        ME      RMSE       MAE         MPE      MAPE      MASE
## Training set 3.875562e-05 0.1354177 0.1057449 -0.03955735  1.456789 0.2060922
## Test set     8.593143e-01 1.5585105 1.0433656  9.62750723 13.220081 2.0334735
##                     ACF1 Theil's U
## Training set 0.004892823        NA
## Test set     0.922448265   1.81467
```

```
accuracy(armax_fc_4,  test)
```

```
##                        ME      RMSE       MAE         MPE      MAPE      MASE
## Training set 4.726928e-05 0.1354086 0.1057373 -0.03931746  1.456751 0.2060772
## Test set     8.631594e-01 1.5626040 1.0458233  9.68113517 13.247136 2.0382634
##                     ACF1 Theil's U
## Training set 0.005812298        NA
## Test set     0.922361435  1.819509
```

```
accuracy(armax_fc_5,  test)
```

```
##                        ME      RMSE       MAE        MPE      MAPE      MASE
## Training set 3.397578e-05 0.1353956 0.1057526 -0.0392822  1.457091 0.2061071
## Test set     8.622335e-01 1.5621837 1.0455136  9.6661733 13.244029 2.0376599
##                     ACF1 Theil's U
## Training set 0.004872423        NA
## Test set     0.922355289  1.818847
```

```
accuracy(armax_fc_6,  test)
```

```
##                       ME      RMSE       MAE         MPE      MAPE      MASE
## Training set -0.00156491 0.1518834 0.1137554 -0.04820475  1.577104 0.2217042
## Test set      1.08976049 1.7015992 1.1555533 13.25175289 14.565971 2.2521225
##                    ACF1 Theil's U
## Training set 0.04305154        NA
## Test set     0.92221592   2.02556
```

```
accuracy(armax_fc_7,  test)
```

```
##                        ME      RMSE      MAE         MPE      MAPE      MASE
## Training set -0.0001000522 0.1506763 0.115562 -0.02348118  1.605761 0.2252253
## Test set      1.1895153796 1.8120552 1.234105 14.64935697 15.546556 2.4052169
##                     ACF1 Theil's U
## Training set -0.07816229        NA
## Test set      0.92015991  2.167033
```

```
accuracy(armax_fc_8,  test)
```

```
##                       ME      RMSE      MAE         MPE      MAPE      MASE
## Training set -0.001152949 0.1514237 0.114025 -0.04130762  1.581546 0.2222296
## Test set      1.113211874 1.7274201 1.173844 13.58042478 14.793247 2.2877695
##                    ACF1 Theil's U
## Training set 0.02839724        NA
## Test set     0.92173437   2.05852
```

```
accuracy(armax_fc_9,  test)
```

```
##                       ME      RMSE       MAE         MPE      MAPE     MASE
## Training set -0.001796941 0.1521639 0.1137373 -0.05221515  1.576593 0.221669
## Test set      1.074327211 1.6858393 1.1440856 13.03082908 14.422587 2.229772
##                    ACF1 Theil's U
## Training set 0.04055114        NA
## Test set     0.92249261  2.005039
```

```
accuracy(armax_fc_10, test)
```

```
##                       ME      RMSE       MAE         MPE      MAPE     MASE
## Training set -0.001902557 0.1522569 0.1136896 -0.05395923  1.575819 0.221576
## Test set      1.069625721 1.6806591 1.1405686 12.96501362 14.379850 2.222918
##                    ACF1 Theil's U
## Training set 0.03755019        NA
## Test set     0.92259536  1.998438
```

```
accuracy(armax_fc_11, test)
```

```
##                         ME      RMSE       MAE         MPE      MAPE      MASE
## Training set -0.0002100046 0.1346365 0.1047598 -0.04371199  1.442263 0.2041722
## Test set      0.7386363216 1.4952357 0.9964941  7.72621210 12.727591 1.9421231
##                     ACF1 Theil's U
## Training set 0.003474615        NA
## Test set     0.923733039  1.732153
```

```
accuracy(armax_fc_12, test)
```

```
##                         ME      RMSE       MAE        MPE      MAPE      MASE
## Training set -0.0003232806 0.1353387 0.1053025 -0.0408776  1.450058 0.2052299
## Test set      0.7946366051 1.5561737 1.0313010  8.4960542 13.096121 2.0099602
##                    ACF1 Theil's U
## Training set -0.1016568        NA
## Test set      0.9223600  1.799277
```

```
accuracy(armax_fc_13, test)
```

```
##                         ME      RMSE       MAE         MPE     MAPE      MASE
## Training set -0.0002168582 0.1346202 0.1047734 -0.04327768  1.44276 0.2041987
## Test set      0.7438917697 1.5010045 0.9997073  7.79814279 12.76102 1.9483855
##                     ACF1 Theil's U
## Training set 0.003128014        NA
## Test set     0.923585326  1.738481
```

```
accuracy(armax_fc_14, test)
```

```
##                         ME      RMSE       MAE         MPE      MAPE      MASE
## Training set -0.0002119412 0.1346278 0.1047624 -0.04356564  1.442416 0.2041773
## Test set      0.7401124856 1.4968339 0.9973892  7.74649181 12.736926 1.9438677
##                     ACF1 Theil's U
## Training set 0.004391877        NA
## Test set     0.923691439  1.733905
```

```
accuracy(armax_fc_15, test)
```

```
##                        ME     RMSE       MAE        MPE      MAPE      MASE
## Training set -0.000212497 0.134623 0.1047627 -0.0434849  1.442467 0.2041778
## Test set      0.741254473 1.498006 0.9980503  7.7624041 12.743772 1.9451561
##                     ACF1 Theil's U
## Training set 0.004593337        NA
## Test set     0.923662806  1.735196
```

```
plot(armax_fc_15,
     main = "ARMAX (0,1,2) Forecast: Unemployment Rate",
     ylab = "Unemployment Rate (%)",
     xlab = "Time",
     sub = "Figure 5: ARIMAX forecast vs. actual unemployment rate")

lines(test, col = "red")
```

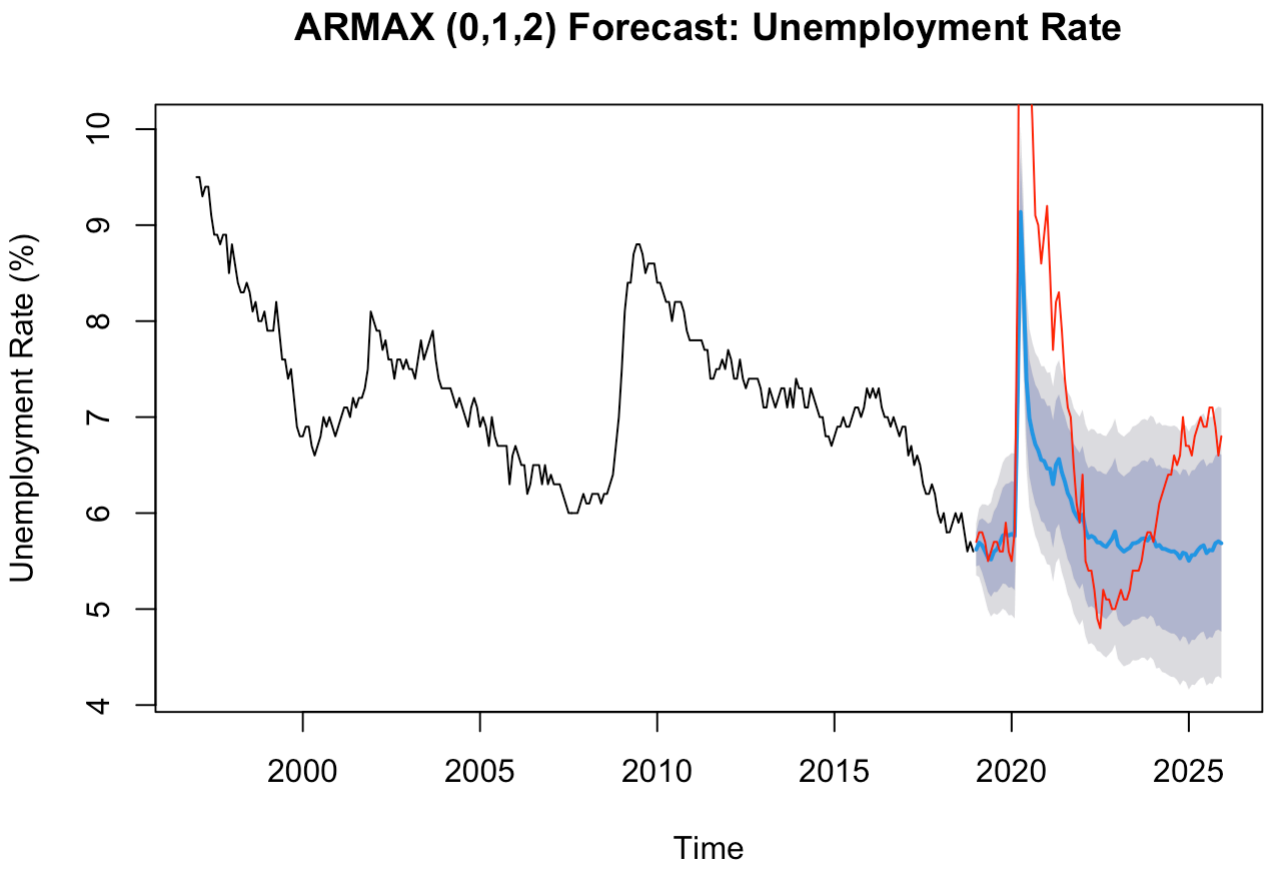
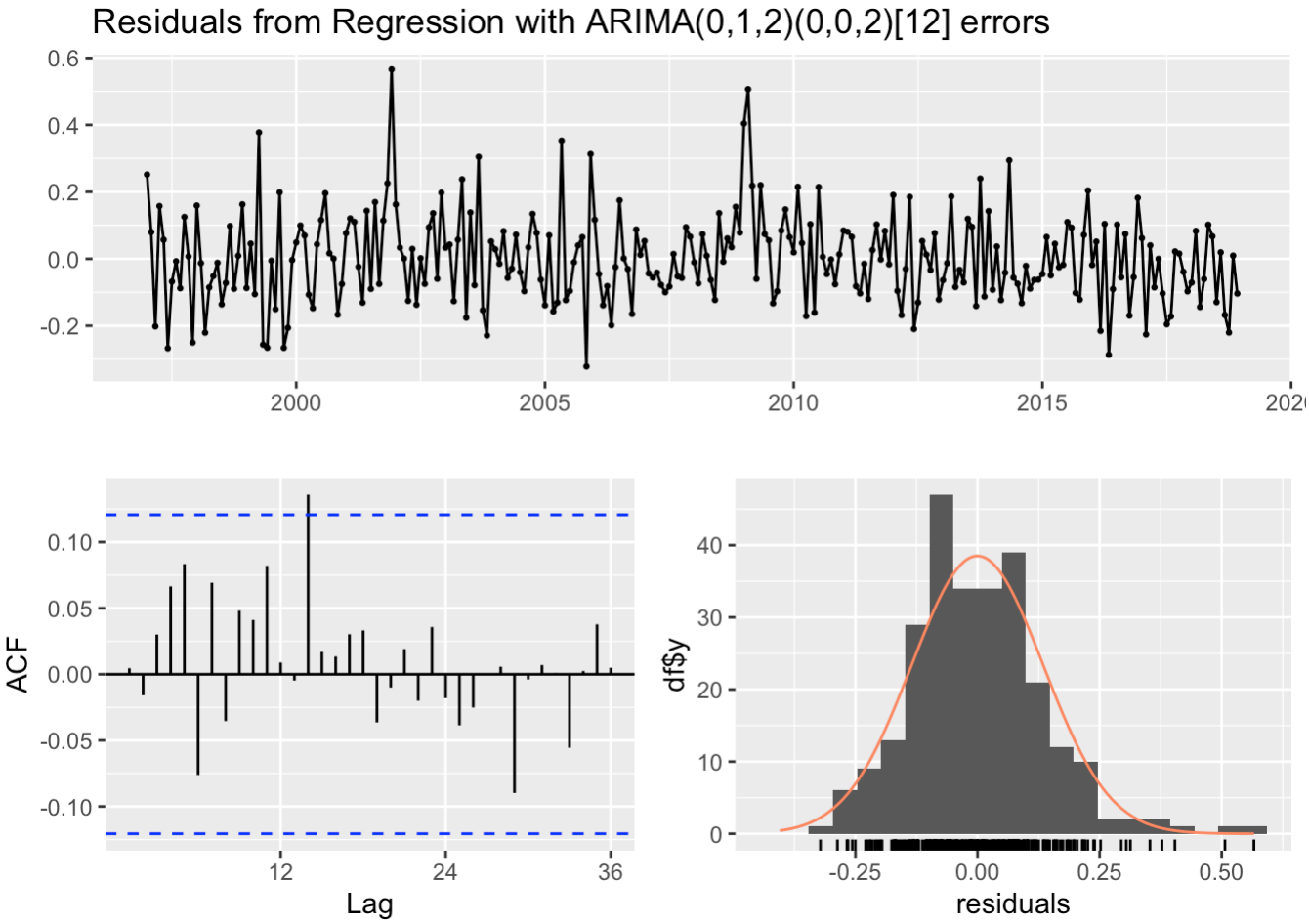


Figure 5: ARIMAX forecast vs. actual unemployment rate

```
checkresiduals(armax_fc_15)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from Regression with ARIMA(0,1,2)(0,0,2)[12] errors
## Q* = 16.615, df = 20, p-value = 0.6778
##
## Model df: 4.   Total lags used: 24
```

```
Box.test(residuals(armax_fc_15), lag = 20, type = "Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  residuals(armax_fc_15)
## X-squared = 15.927, df = 20, p-value = 0.7212
```